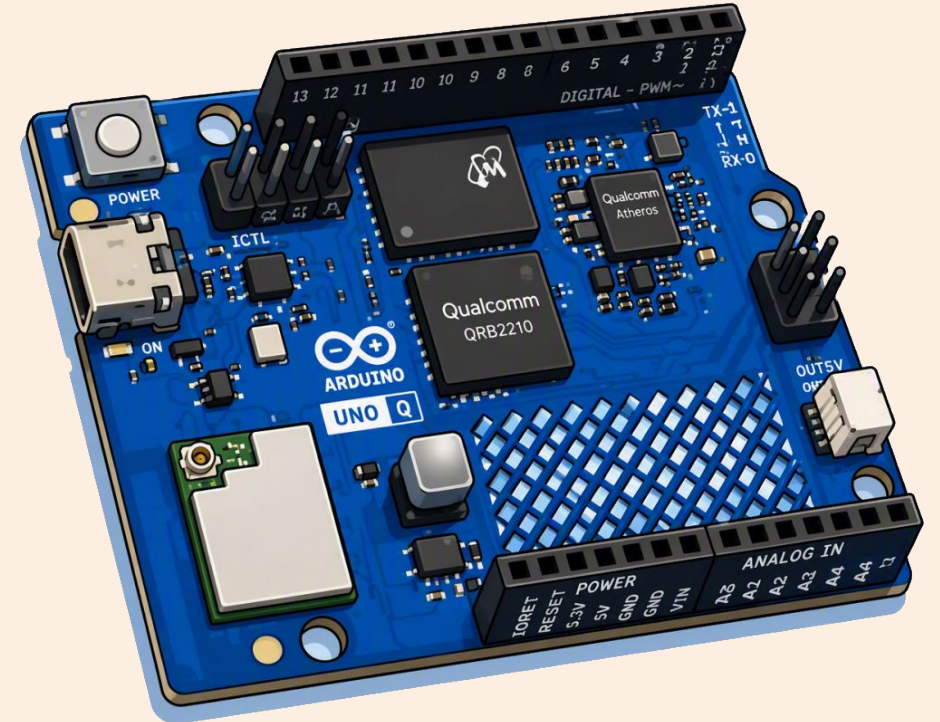


Setting Up the Arduino UNO Q

A terminal-first workflow: ADB, Wi-Fi, SSH

Debian Linux + Zephyr RTOS
no GUI · no monitor · no keyboard

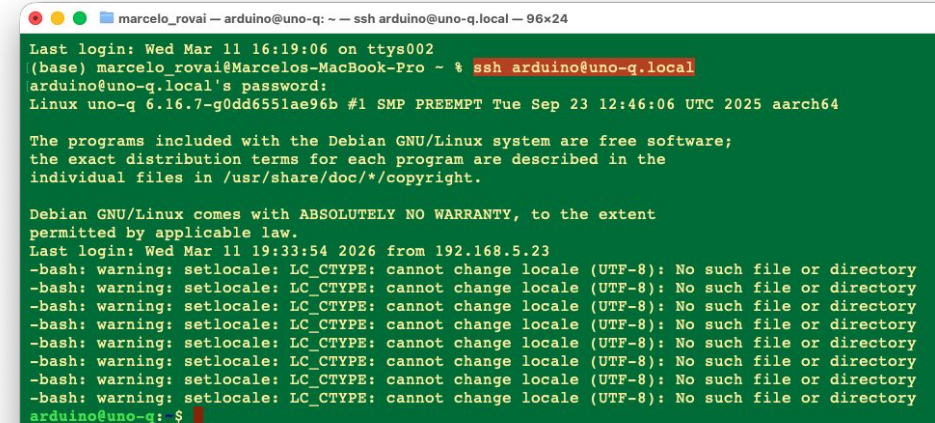
Marcelo Rovai · UNIFEI



WHERE WE ARE GOING

By the end of this chapter you will be here.

An SSH session on the board and a dual-brain app
running from one command.



```
marcelo_rovai — arduino@uno-q: ~ — ssh arduino@uno-q.local — 96x24
Last login: Wed Mar 11 16:19:06 on ttys002
(base) marcelo_rovai@Marcelos-MacBook-Pro ~ % ssh arduino@uno-q.local
arduino@uno-q.local's password:
Linux uno-q 6.16.7-g0dd6551ae96b #1 SMP PREEMPT Tue Sep 23 12:46:06 UTC 2025 aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Mar 11 19:33:54 2026 from 192.168.5.23
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
arduino@uno-q: $
```

Seven stages, one shell at the end

- 1 The board**

Two processors, one UNO footprint, an RPC bridge between them.
- 2 ADB**

One tool on your computer, talking over the USB-C cable.
- 3 Flash**

Optional, but it makes a whole classroom behave the same way.
- 4 Connect**

First Linux shell on the board — and change the password.
- 5 Wi-Fi + SSH**

Cut the cable. From here the board is a network machine.
- 6 Files**

`scp` for scripts, `SFTP` for week one, `git` for anything you iterate on.
- 7 Run apps**

`arduino-app-cli`: one command drives both processors.

STAGE 1 OF 8

What the board actually is

Two processors, one UNO footprint, and an RPC bridge between them.

1

ARCHITECTURE

One board, two processors

MPU · MICROPROCESSOR UNIT

Qualcomm QRB2210

Quad-core Cortex-A53 @ 2.0 GHz

Adreno 702 GPU · dual ISP

Runs Debian Linux — Python, AI inference, networking.

MCU · MICROCONTROLLER UNIT

ST STM32U585

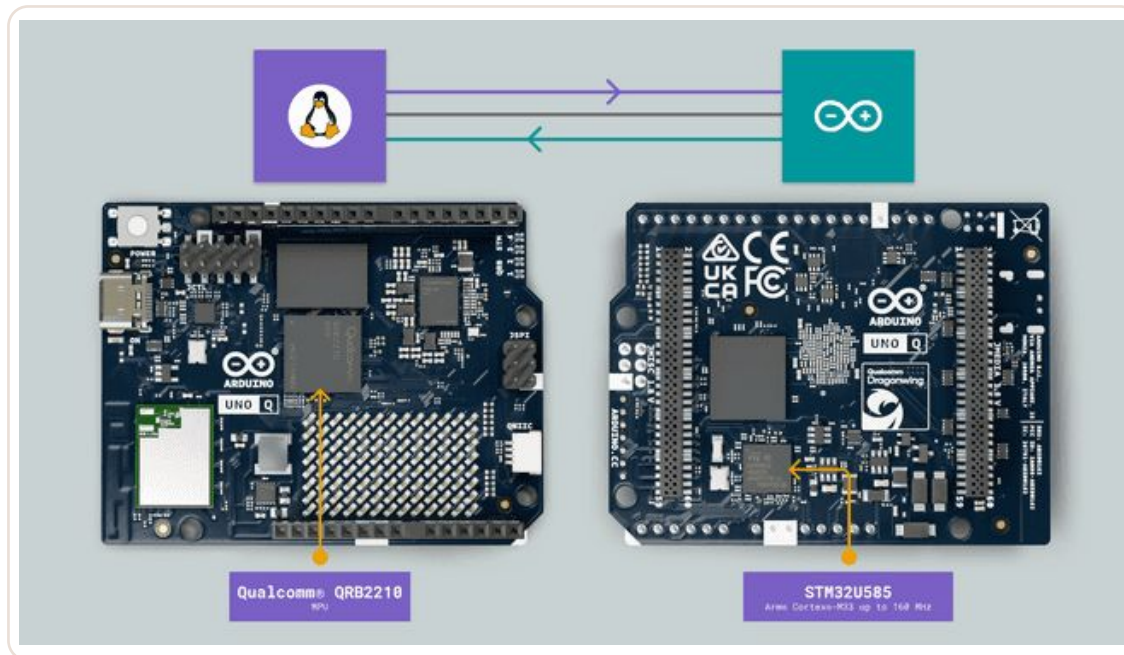
Cortex-M33 @ 160 MHz

Runs the Arduino Core on Zephyr — deterministic, real-time hardware control.

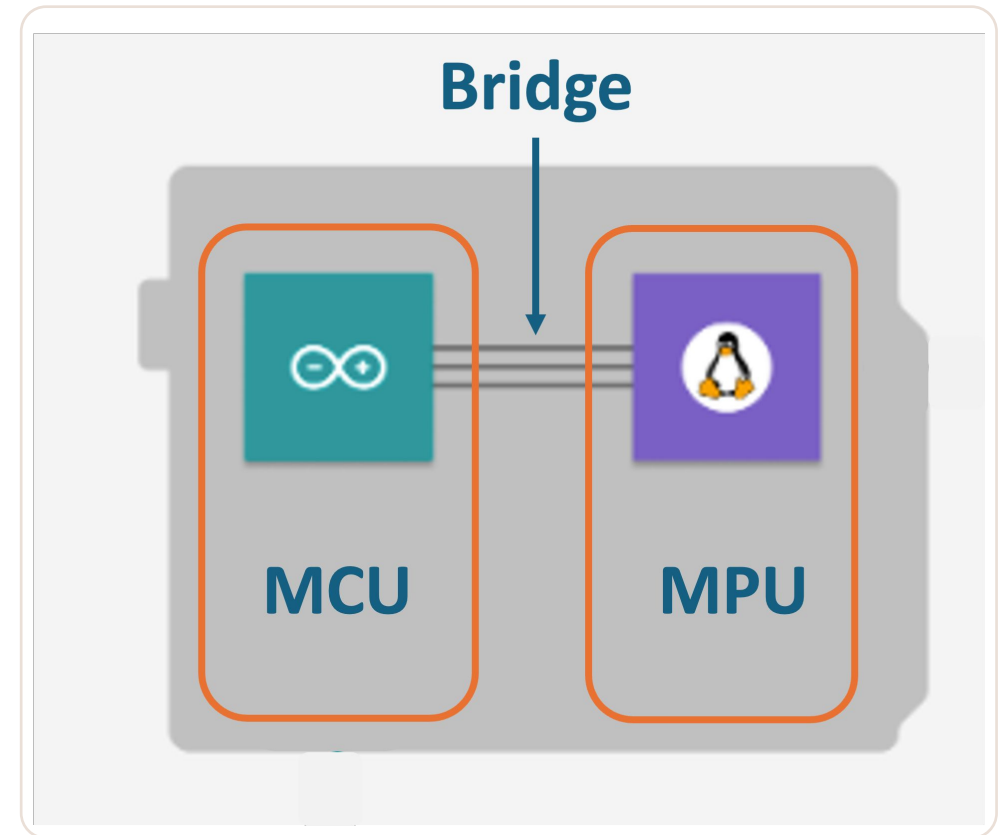
Bridge — Arduino's RPC library — lets Python on the MPU call functions in the sketch on the MCU, and back.

ARCHITECTURE

The bridge between them



UNO Q Architecture



Python on the MPU $\rightleftharpoons$ Arduino sketch on the MCU

WHICH ONE TO BUY

Two variants, one architecture

ENTRY

2 GB

LPDDR4X · 16 GB eMMC

Headless SSH development,
lightweight edge AI, TinyML.

COMFORTABLE

4 GB

LPDDR4X · 32 GB eMMC

SBC mode with a display, larger AI
models, multitasking.

BOTH

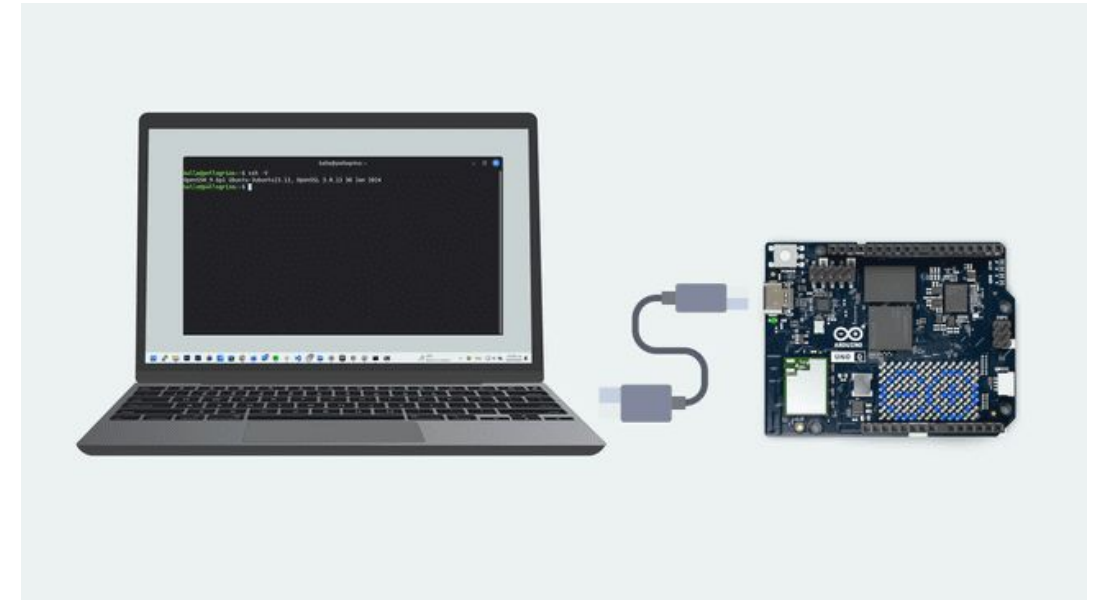
IdenticalSame processors · Wi-Fi 5 + BT 5.1
· USB-C · UNO headers · Qwiic ·
8×13 LED matrix.

RAM is the only difference that matters — and it determines whether generative AI is comfortable.

THE CABLE

The first thing that goes wrong is the cable.

One USB-C port carries power, data, and video. It must be a data cable — not a charge-only one. Some hubs and USB-C adapters are not recognized at all.



Accessing the board shell

APPROACH

Why terminal-first, and not the App Lab GUI

1

Full control

You work inside the board's Debian environment — nothing hidden behind an abstraction.

2

Code lives with you

Projects stay on your machine under Git, not locked inside the board.

3

Transferable skills

Linux, SSH and SCP are what real edge-AI deployments actually use.

4

Honest about failure

You see the actual error, not a spinner. That is where the learning is.

Also: scriptable, and editor-agnostic — VS Code, Vim, anything.

STAGE 2 OF 8

Host setup: ADB

One tool on your computer opens a shell on the board over the USB-C cable.

2

BEFORE YOU START

What you need

HARDWARE

On the bench

Arduino UNO Q (2 GB or 4 GB)

USB-C data cable

Host computer — Linux, macOS or Windows

NETWORK

Same one, both ends

A Wi-Fi network the board and your computer can both reach.

You will need the SSID and password.

HOST SOFTWARE

Three commands

adb — the first headless connection

ssh — remote access over Wi-Fi

scp — file transfer

INSTALL

Getting ADB onto your computer

```
# Linux — Ubuntu / Debian
```

```
sudo apt update
```

```
sudo apt install android-tools-adb
```

```
# macOS — Homebrew
```

```
brew install android-platform-tools
```

```
# Windows
```

```
Download SDK Platform-Tools, extract to C:\platform-tools,  
then add that folder to your PATH
```

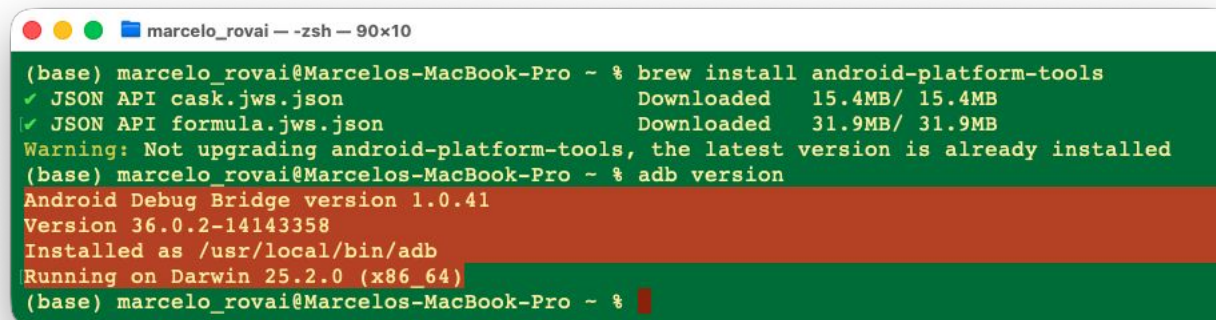
VERIFY

Check ADB before you plug anything in

```
$ adb version
```

```
Android Debug Bridge version 1.0.41
```

Run this with the board DISCONNECTED. If it fails here, nothing downstream will work — and you will spend an hour blaming the board.

A terminal window titled 'marcelo_rovai - zsh - 90x10' with a green background. It shows the command 'brew install android-platform-tools' being executed, followed by progress bars for downloading JSON API files. A warning message states that the latest version is already installed. Then, the command 'adb version' is run, displaying 'Android Debug Bridge version 1.0.41', 'Version 36.0.2-14143358', 'Installed as /usr/local/bin/adb', and 'Running on Darwin 25.2.0 (x86_64)'.

```
(base) marcelo_rovai@Marcelos-MacBook-Pro ~ % brew install android-platform-tools
✓ JSON API cask.jws.json           Downloaded 15.4MB/ 15.4MB
✓ JSON API formula.jws.json        Downloaded 31.9MB/ 31.9MB
Warning: Not upgrading android-platform-tools, the latest version is already installed
(base) marcelo_rovai@Marcelos-MacBook-Pro ~ % adb version
Android Debug Bridge version 1.0.41
Version 36.0.2-14143358
Installed as /usr/local/bin/adb
Running on Darwin 25.2.0 (x86_64)
(base) marcelo_rovai@Marcelos-MacBook-Pro ~ %
```

STAGE 3 OF 8

Flashing the latest image

Optional — but essential if you want a whole classroom behaving the same way.

3

RECOMMENDED FIRST STEP

Why flash a board that already boots

1

Random restarts

The old image has no ZRAM compression — apps restart unpredictably under memory pressure.

2

ADB defaults to root

A security problem, and the wrong habit to learn in the first minute.

3

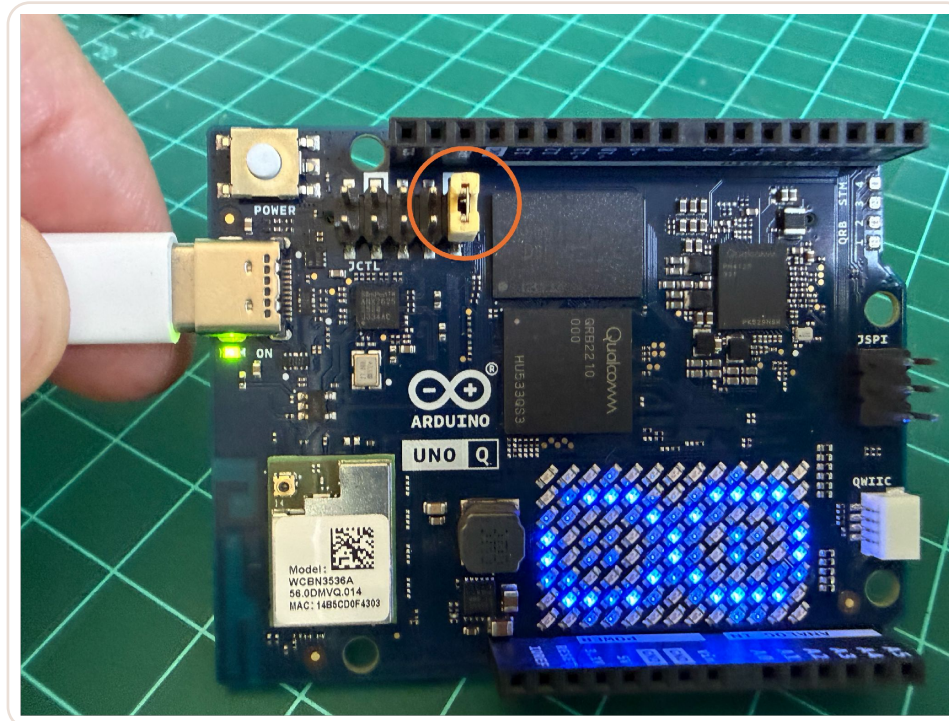
No HDMI audio

Simply missing from the shipped image.

Flashing erases everything — the board goes back to factory state. The tool downloads ~2.3 GB, so use a good connection, or download once and share the image.

STEP 1 OF 3

Jumper the JCTL header — this is EDL mode



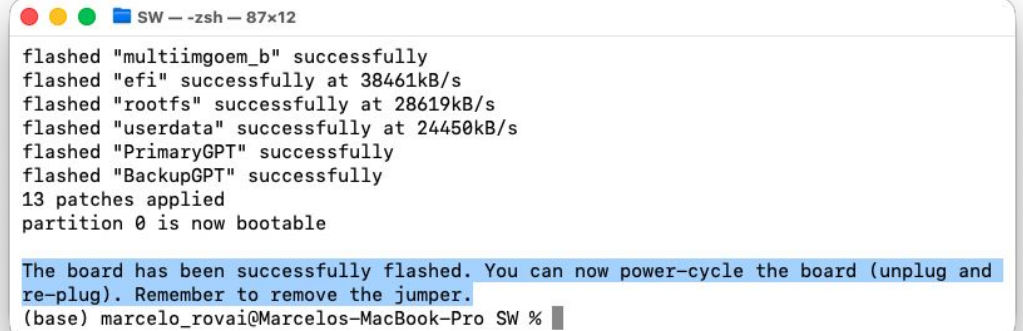
The LED matrix loops the Arduino logo — you are in EDL mode

Board powered OFF. Short the two pins FURTHEST from the USB-C connector, then plug the cable in. Jumper first, cable second.

STEP 2 OF 3

Run the flasher

```
$ ./arduino-flasher-cli flash latest  
Download image? [y/N] y  
Downloading ~2.3 GB ...  
Flashing ...
```



A terminal window titled "SW - zsh - 87x12" showing the output of the flashing process. The output lists several partitions being flashed successfully with their respective speeds: multiimgoem_b, efi, rootfs, userdata, PrimaryGPT, and BackupGPT. It also mentions that 13 patches were applied and that partition 0 is now bootable. A blue highlight covers the final instruction: "The board has been successfully flashed. You can now power-cycle the board (unplug and re-plug). Remember to remove the jumper." The prompt at the bottom is "(base) marcelo_rovai@Marcelos-MacBook-Pro SW %".

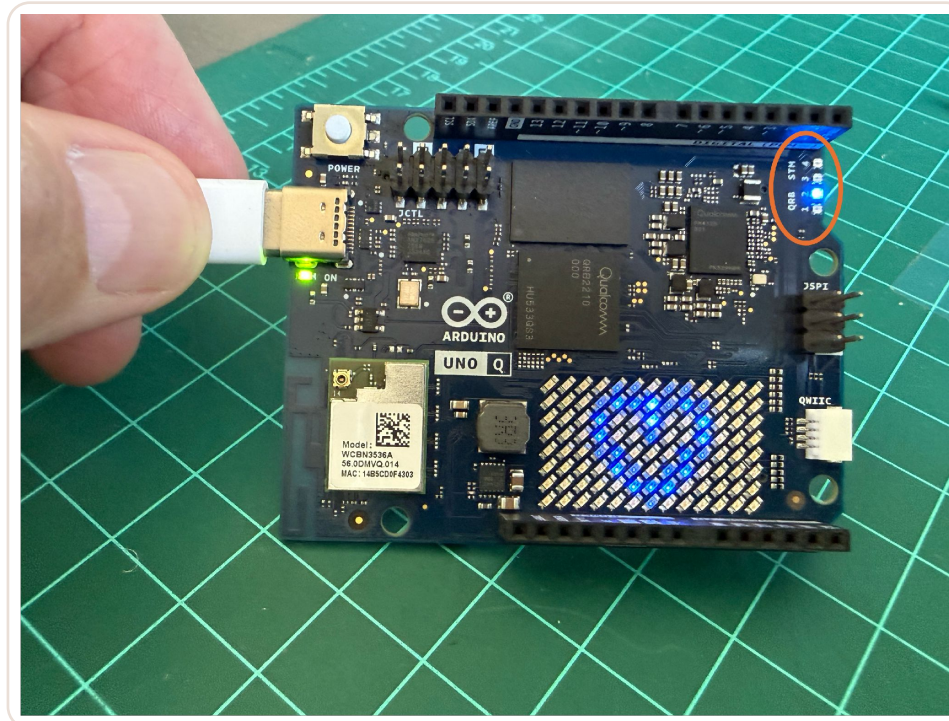
```
flashed "multiimgoem_b" successfully  
flashed "efi" successfully at 38461kB/s  
flashed "rootfs" successfully at 28619kB/s  
flashed "userdata" successfully at 24450kB/s  
flashed "PrimaryGPT" successfully  
flashed "BackupGPT" successfully  
13 patches applied  
partition 0 is now bootable  
  
The board has been successfully flashed. You can now power-cycle the board (unplug and  
re-plug). Remember to remove the jumper.  
(base) marcelo_rovai@Marcelos-MacBook-Pro SW %
```

Flashing in progress

Do not disconnect the cable or interrupt the process. Get arduino-flasher-cli from the [Arduino Software Download page](#) or its [GitHub releases](#).

STEP 3 OF 3

Remove the jumper and reboot



Logo animation ends in a heart, blue LED 2 lit — the flash worked

Disconnect · remove the jumper · reconnect.

STAGE 4 OF 8

First connection over ADB

Plug in the cable, open a shell, change the password.

4

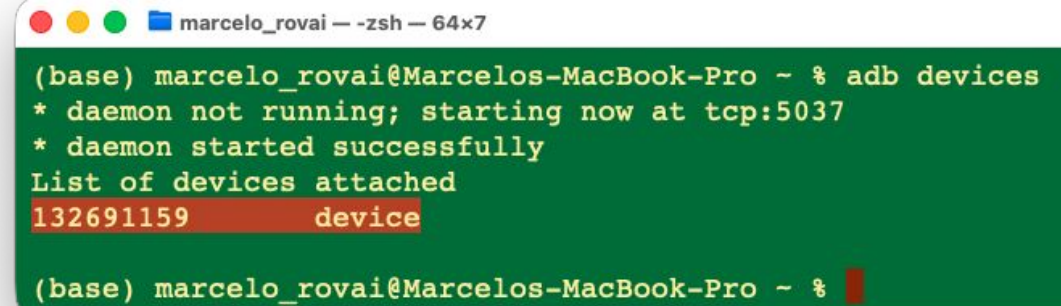
HEADLESS BRING-UP

Is the board there?

```
$ adb devices
```

```
List of devices attached
```

```
XXXXXXXXXX    device
```



```
marcelo_rovai — zsh — 64x7  
(base) marcelo_rovai@Marcelos-MacBook-Pro ~ % adb devices  
* daemon not running; starting now at tcp:5037  
* daemon started successfully  
List of devices attached  
132691159      device  
(base) marcelo_rovai@Marcelos-MacBook-Pro ~ %
```

adb devices — the board is seen

Do not debug anything else until this line shows a device.

IF THE LIST COMES BACK EMPTY

Check these three, in this order

1

The cable

Is it a data cable, or a charge-only one? This is the answer about half the time.

2

The port

Plug directly into the computer. Hubs and USB-C adapters are frequently not recognised.

3

udev rules

On Linux only — without them the device is there but you have no permission to see it.

Order matters: cable, then port, then udev. Working down the list beats guessing.

YOU ARE ON THE BOARD

Open a shell and change the password

```
$ adb shell
```

```
arduino@uno-q:~$
```

```
$ sudo passwd arduino
```

Default credentials are arduino / arduino. Change the password NOW — you need the new one for SSH, and the board will keep nagging you until you do.

KNOW WHAT YOU ARE WORKING WITH

Four commands worth running once

```
$ cat /etc/os-release      # which Debian
$ lscpu                   # the four A53 cores
$ free -h                 # RAM and swap
$ df -h                   # eMMC space left
```

Run these on day one. Knowing the real numbers explains a lot of later behavior.

STAGE 5 OF 8

Wi-Fi and SSH

Configure the network from inside the ADB shell, then drop the cable.

5

NETWORKING

Join a Wi-Fi network with nmcli

```
# inside the ADB shell
$ nmcli dev wifi list

$ sudo nmcli dev wifi connect "YOUR_SSID" \
    password "YOUR_PASSWORD"
```



A terminal window titled 'marcelo_rovai -- arduino@uno-q: ~ -- ssh arduino@uno-q.local -- 68x7' showing the output of the 'nmcli device status' command. The output is a table with four columns: DEVICE, TYPE, STATE, and CONNECTION. The first row, 'wlan0', is highlighted in red and shows 'wifi', 'connected', and 'ROVAI TIMECAP'. The other rows are 'lo' (loopback, connected (externally), lo), 'docker0' (bridge, connected (externally), docker0), and 'p2p-dev-wlan0' (wifi-p2p, disconnected, --).

DEVICE	TYPE	STATE	CONNECTION
wlan0	wifi	connected	ROVAI TIMECAP
lo	loopback	connected (externally)	lo
docker0	bridge	connected (externally)	docker0
p2p-dev-wlan0	wifi-p2p	disconnected	--

nmcli device status

Quote the SSID if it contains spaces. Some networks hide their 5 GHz SSID — check the band.

NETWORKING

Confirm, and get the address you need

```
$ nmcli device status  
wlan0    wifi    connected  
  
$ hostname -I  
192.168.5.85
```



A terminal window titled 'marcelo_rovai — arduino@uno-q: ~ — ssh arduino@uno-q.local — 68x10'. It shows the output of 'nmcli device status' and 'hostname -I'. The IP address '192.168.5.85' is highlighted in red in the original image.

DEVICE	TYPE	STATE	CONNECTION
wlan0	wifi	connected	ROVAI TIMECAP
lo	loopback	connected (externally)	lo
docker0	bridge	connected (externally)	docker0
p2p-dev-wlan0	wifi-p2p	disconnected	--

hostname -I output: 172.17.0.1 192.168.5.85 fde3:6154:bba3:1:82b:9faa:c2f:8cae fd4d:57e8:907a:44b0:f61d:8bb9:3332:2972

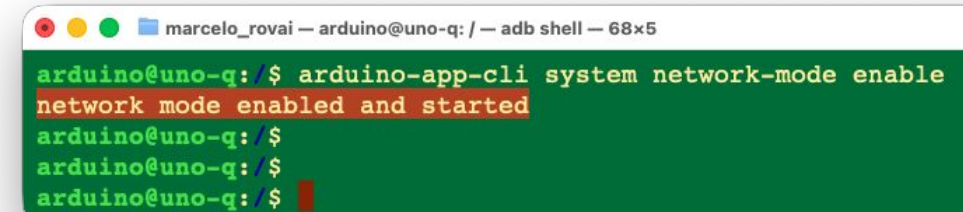
Connected — write this address down

Better than writing it down: take the MAC from `ip addr show wlan0` and give the board a static lease on your router, so the address survives reboots.

REMOTE ACCESS

Enable SSH — the step everyone forgets

```
# on the board, in the ADB shell
$ arduino-app-cli system network-mode enable
$ exit
```



```
marcelo_rovai — arduino@uno-q: / — adb shell — 68x5
arduino@uno-q: $ arduino-app-cli system network-mode enable
network mode enabled and started
arduino@uno-q: $
arduino@uno-q: $
arduino@uno-q: $
```

network-mode enable

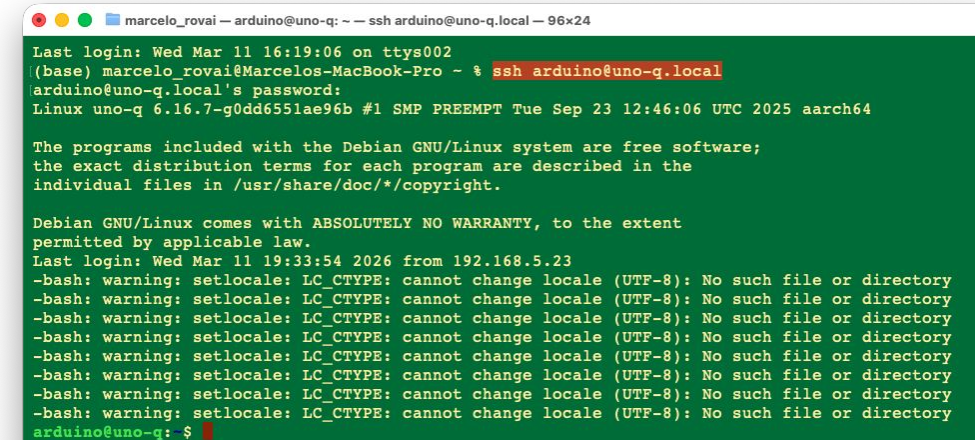
This is the single most common place people get stuck. SSH exists on the board, but it is not active until this command runs.

REMOTE ACCESS

Drop the cable

```
# from your host computer now
$ ssh arduino@192.168.5.85

# or, on networks with mDNS
$ ssh arduino@uno-q.local
```



```
marcelo_rovai — arduino@uno-q: ~ — ssh arduino@uno-q.local — 96x24
Last login: Wed Mar 11 16:19:06 on ttys002
(base) marcelo_rovai@Marcelos-MacBook-Pro ~ % ssh arduino@uno-q.local
arduino@uno-q.local's password:
Linux uno-q 6.16.7-g0dd6551ae96b #1 SMP PREEMPT Tue Sep 23 12:46:06 UTC 2025 aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Mar 11 19:33:54 2026 from 192.168.5.23
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
-bash: warning: setlocale: LC_CTYPE: cannot change locale (UTF-8): No such file or directory
arduino@uno-q:~$
```

Connected over Wi-Fi — the cable is just power now

Password rejected? Go back in over ADB — adb shell , then sudo passwd arduino — exit, and retry.

HOUSEKEEPING

Update the system once

```
$ sudo apt update && sudo apt upgrade  
$ sudo reboot
```

Do this now, over Wi-Fi, before installing anything else.

KNOW YOUR BOARD

htop: 2 GB versus 4 GB, side by side

```
marcelo_rovai — arduino@uno-q: ~ — ssh arduino@uno-q.local — 84x22

0[          0.0%] Tasks: 79, 174 thr, 119 kthr; 1 running
1[          0.6%] Load average: 0.09 0.06 0.01
2[          3.2%] Uptime: 21:14:22
3[          0.6%]
Mem[|||||] 1.07G/1.70G
Swp[|||||] 151M/870M

Main I/O
PID USER PRI NI VIRT RES SHR S CPU%-MEM% TIME+ Command
4580 arduino 20 0 7900 3656 2508 R 2.6 0.2 0:01.96 htop
856 lightdm 20 0 797M 40516 22060 S 0.6 2.3 1:37.00 /usr/sbin/lightdm
1962 arduino 20 0 51.8G 559M 18348 S 0.6 32.1 4:41.49 /home/arduino/.vs
1 root 20 0 25580 12940 8248 S 0.0 0.7 0:10.16 /sbin/init
238 root 20 0 47884 9328 7892 S 0.0 0.5 0:02.34 /usr/lib/systemd/
278 systemd-t 20 0 92496 6764 5632 S 0.0 0.4 0:00.81 /usr/lib/systemd/
287 root 20 0 36876 9976 6000 S 0.0 0.6 0:01.46 /usr/lib/systemd/
288 systemd-t 20 0 92496 6764 0 S 0.0 0.4 0:00.00 /usr/lib/systemd/
414 root 20 0 302M 6580 5784 S 0.0 0.4 0:00.22 /usr/libexec/acco
417 avahi 20 0 6700 3568 2668 S 0.0 0.2 1:41.59 avahi-daemon: run
419 root 20 0 13008 4704 4044 S 0.0 0.3 0:00.16 /usr/libexec/blue

F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice -F8Nice +F9Kill F10Quit
```

2 GB variant

```
arduino-flasher-cli-0.5.1-darwin-arm64 — arduino@uno-q: ~ — ssh arduino@192.168.5.114 — 91x24

0[          0.0%] Tasks: 57, 118 thr, 125 kthr; 1 running
1[          0.0%] Load average: 0.14 0.17 0.15
2[          2.6%] Uptime: 00:23:05
3[          0.0%]
Mem[|||||] 417M/3.58G
Swp[|||||] 0K/1.79G

Main I/O
PID USER PRI NI VIRT RES SHR S CPU%-MEM% TIME+ Command
3055 arduino 20 0 7836 3612 2492 R 2.0 0.1 0:01.32 htop
1 root 20 0 26428 15644 9892 S 0.0 0.4 0:32.66 /sbin/init
258 root 20 0 62556 9020 7600 S 0.0 0.2 0:00.81 /usr/lib/systemd/systemd
299 systemd-ti 20 0 92504 7564 6432 S 0.0 0.2 0:00.30 /usr/lib/systemd/systemd
308 root 20 0 36928 12420 7284 S 0.0 0.3 0:01.25 /usr/lib/systemd/systemd
309 systemd-ti 20 0 92504 7564 0 S 0.0 0.2 0:00.18 /usr/lib/systemd/systemd
449 root 20 0 302M 6852 6048 S 0.0 0.2 0:00.20 /usr/libexec/accounts-da
453 root 20 0 13016 5948 5284 S 0.0 0.2 0:00.15 /usr/libexec/bluetooth/b
454 root 20 0 6988 2612 2372 S 0.0 0.1 0:00.04 /usr/sbin/cron -f
455 messagebus 20 0 9620 5528 3780 S 0.0 0.1 0:09.92 /usr/bin/dbus-daemon --s
459 root 20 0 302M 6852 0 S 0.0 0.2 0:00.00 /usr/libexec/accounts-da
460 root 20 0 302M 6852 0 S 0.0 0.2 0:00.00 /usr/libexec/accounts-da
462 polkitd 20 0 374M 9728 7172 S 0.0 0.3 0:00.42 /usr/lib/polkit-1/polkit

F1Help F2Setup F3Search F4Filter F5Tree F6SortBy F7Nice -F8Nice +F9Kill F10Quit
```

4 GB variant

Both the RAM and the swap are doubled on the 4 GB board.

STAGE 6 OF 8

Getting code onto the board

6

scp for scripts, SFTP for week one, git for anything you iterate on.

MOVING CODE

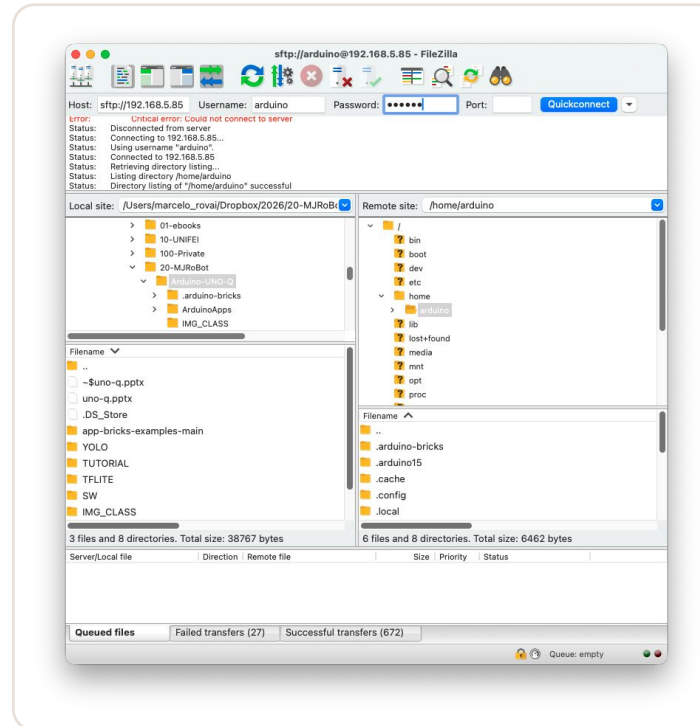
scp — the one that scripts

```
$ cd <your-project-folder>  
$ scp -r * arduino@192.168.5.85:~/ArduinoApps/<folder>/
```

Because it is a plain command, it drops straight into a Makefile or a deploy script. For repeated syncs use `rsync` — it only sends what changed, which matters over Wi-Fi.

MOVING CODE

SFTP — the one they will actually use



FileZilla: both filesystems side by side

Host field: sftp://192.168.5.85 · board credentials · Quickconnect.

STAGE 7 OF 8

Running things: arduino-app-cli



One command orchestrates the Python container and the MCU sketch.

AN IMPORTANT DISTINCTION

Two classes of app

READ-ONLY, CLI-MANAGED

examples:*

Live in `/var/lib/arduino-app-cli` — not in your home directory, which is why `ls` never shows them.

```
arduino-app-cli app start examples:blink
```

YOURS, EDITABLE

user:*

Live in `~/ArduinoApps/`. The only place `ls` shows anything, and the only place you edit.

```
arduino-app-cli app new "Blink"
```

This trips everyone up: `app list` shows examples you cannot find with `ls`. They are managed by the CLI.

BEFORE YOU EXPLORE

Pull the latest components

```
$ arduino-app-cli system update
```

This takes a long time. Start it before a break, not in the middle of a demo.

```
marcelo_rovai — arduino@uno-q: ~ — ssh arduino@uno-q.local — 112x32
arduino@uno-q: $ arduino-app-cli app list
```

ID	NAME	ICON	STATUS	EXAMPLE
examples:air-quality-monitoring	Air quality on LED matrix		uninitialized	true
examples:anomaly-detection	Concrete crack detector		uninitialized	true
examples:audio-classification	Glass breaking sensor		uninitialized	true
examples:bedtime-story-teller	Bedtime story teller		uninitialized	true
examples:blink	Blink LED		uninitialized	true
examples:blink-with-ui	Blink LED with UI		uninitialized	true
examples:cloud-blink	Blinking LED from Arduino Cloud		uninitialized	true
examples:code-detector	QR and Barcode Scanner		uninitialized	true
examples:color-your-leds	Color your LEDs		uninitialized	true
examples:home-climate-monitoring-and-storage	Home climate monitoring and storage		uninitialized	true
examples:image-classification	Classify images		uninitialized	true
examples:keyword-spotting	Hey Arduino!		uninitialized	true
examples:led-matrix-painter	Led Matrix Painter		uninitialized	true
examples:mascot-jump-game	Mascot Jump Game		uninitialized	true
examples:mobile-video-generic-object-detection	Detect Objects on Smartphone Camera		uninitialized	true
examples:object-detection	Detect objects on images		uninitialized	true
examples:object-hunting	Object Hunting		uninitialized	true
examples:real-time-accelerometer	Real-time Accelerometer		uninitialized	true
examples:system-resources-logger	System resources logger		uninitialized	true
examples:theremin	Theremin simulator		uninitialized	true
examples:unoq-pin-toggle	UNO Q Pin Toggle		uninitialized	true
examples:vibration-anomaly-detection	Fan Vibration Monitoring		uninitialized	true
examples:video-face-detection	Face Detector on Camera		uninitialized	true
examples:video-generic-object-detection	Detect Objects on Camera		uninitialized	true
examples:video-person-classification	Person classifier on camera		uninitialized	true
examples:weather-forecast	Weather forecast on LED matrix		uninitialized	true
user:Blink	Blink LED		uninitialized	false
user:my-blink	Blink LED		uninitialized	false
user:test	test		uninitialized	false

```
arduino@uno-q: $
```

app list — both classes, one listing

THE FIRST DUAL-BRAIN APP

Start it

```
$ arduino-app-cli app start  
examples:blink
```

The FIRST run takes several minutes — it downloads Arduino libraries and builds the Python container. Warn people, or half of them will decide it is broken and reboot.

```
marcelo_rovai — arduino@uno-q: ~ — ssh arduino@uno-q.local — 107x11  
arduino@uno-q: $ arduino-app-cli app start examples:blink  
[INFO] Starting app "Blink LED"  
Progress[preparing]: 0%  
Progress[sketch compiling and uploading]: 0%  
[INFO] Sketch profile configured: Name="default", Port=""  
[INFO] Task Downloading library ArxTypeTraits@0.3.1: 0.00%  
[INFO] Download started: https://downloads.arduino.cc/libraries/github.com/hideakitai/ArxTypeTraits-0.3.1.zip  
[INFO] Download progress: downloaded:19262 total_size:19262  
[INFO] Download completed: success:true  
[INFO] Task : completed
```

•
•
•

```
Progress[python provisioning]: 10%  
[INFO] python downloading  
[INFO] Network local-share-arduino-app-cli-examples-blink_default Creating  
[INFO] Network local-share-arduino-app-cli-examples-blink_default Created  
[INFO] Container local-share-arduino-app-cli-examples-blink-main-1 Creating  
[INFO] Container local-share-arduino-app-cli-examples-blink-main-1 Created  
[INFO] Container local-share-arduino-app-cli-examples-blink-main-1 Starting  
[INFO] Container local-share-arduino-app-cli-examples-blink-main-1 Started  
Progress[]: 100%  
✓ App "Blink LED" started successfully  
arduino@uno-q: $
```

Starting the app

THE FIRST DUAL-BRAIN APP

Watch it, then stop it

```
$ arduino-app-cli app logs  
examples:blink  
  
$ arduino-app-cli app stop  
examples:blink
```



```
marcelo_rovai — arduino@uno-q: ~ — ssh arduino@uno-q.local — 76x6  
arduino@uno-q:~$ arduino-app-cli app logs examples:blink  
[main] Using CPython 3.13.9 interpreter at: /usr/local/bin/python  
[main] Creating virtual environment at: .cache/.venv  
[main] Activating python virtual environment  
[main] ===== App is starting =====  
[main] 2026-03-13 13:26:17.884 INFO - [MainThread] App: App started
```

app logs — the Python side talking

Add --follow to tail the log live.

REFERENCE

The arduino-app-cli commands you will actually use

Command	What it does
<code>arduino-app-cli app new "<name>"</code>	Create a new app skeleton in ~/ArduinoApps/
<code>arduino-app-cli app start <path></code>	Build if needed, then start the app
<code>arduino-app-cli app logs <path></code>	View the Python-side log — add --follow to tail it
<code>arduino-app-cli app stop <path></code>	Stop a running application
<code>arduino-app-cli app list</code>	List installed and running apps, both classes
<code>arduino-app-cli system update</code>	Update the CLI and board components (slow)
<code>arduino-app-cli system cleanup</code>	Remove unused containers — reclaim eMMC space

app start compiles the sketch, flashes the MCU, builds the Python environment and launches the app — both processors, one command.

SURVIVAL KIT

The Linux commands this course leans on

SYSTEM

```
cat /etc/os-release  
lscpu  
free -h  
df -h  
htop
```

NETWORK

```
ip addr  
nmcli device status  
nmcli dev wifi list  
hostname -I
```

PACKAGES

```
sudo apt update  
sudo apt upgrade  
sudo apt install <pkg>  
sudo apt autoremove
```

FILES

```
ls -lah  
cd / mkdir / cp / mv  
rm -rf <dir>  
cat / nano <file>
```

SERVICES

```
sudo systemctl status ssh  
sudo systemctl restart ssh  
sudo reboot  
sudo shutdown -h now
```

CLEANUP

```
df -h  
sudo apt clean  
arduino-app-cli system  
cleanup  
docker system prune -a
```

Standard Debian — anything you know from a Raspberry Pi or a cloud VM applies here unchanged.

STAGE 8 OF 8

When it breaks — and what it is for

8

The failures you will meet in week one, then an honest look at the board's limits.

WEEK ONE

The failures you will actually hit

1

ADB does not see the board

Charge-only cable, a hub in the path, or missing udev rules on Linux.

2

SSH connection refused

network-mode enable was never run — or you are on a different network.

3

SSH password rejected

Reset it over ADB: `adb shell` , then `sudo passwd arduino` .

4

Everything is mysteriously slow

Check `htop`. If swap is in use, close something — the A53s do not recover from thrashing.

Nearly all of these are cable, network, or memory. Teach the diagnosis order and they stop being interesting.

BE REALISTIC

What this board is not good at

COMPUTE

Roughly a Pi 3

Cortex-A53 cores. Models beyond ~2B parameters are impractical. For heavy inference a Pi 5 or an accelerator is still the right answer.

MEMORY & PORTS

Tight, and singular

2 GB is tight for editor + container + model. One USB-C port shares power, data and video — SBC mode needs a hub.

SILICON & SOFTWARE

No NPU, pre-1.0 tools

Inference runs on CPU and GPU. App Lab and the CLI are pre-1.0 — expect rough edges. No built-in camera.

State the ceiling on day one: students who know it design within it, instead of blaming themselves.

CARRY THESE OUT OF CHAPTER 1

Three things

1

The board is two computers

An MPU running Debian and an MCU running Zephyr, talking over Bridge. Knowing which side a problem is on is half of debugging it.

2

The terminal is the interface

ADB to bootstrap, Wi-Fi to cut the cord, SSH from then on. Code lives on your machine under Git; the board just runs it.

3

One CLI drives both processors

app new, app start, app logs, app stop. Every chapter from here uses exactly those four.

RESOURCES

Where to read further

Resource	Where
UNO Q documentation	docs.arduino.cc/hardware/uno-q
UNO Q datasheet (PDF)	docs.arduino.cc/resources/datasheets/ABX00162-datasheet.pdf
Arduino App CLI tutorial	docs.arduino.cc/software/app-lab/tutorials/cli
Arduino App CLI (GitHub)	github.com/arduino/arduino-app-cli
Edge Impulse — UNO Q	docs.edgeimpulse.com/hardware/boards/arduino-uno-q
arduino-flasher-cli releases	github.com/arduino/arduino-flasher-cli/releases

Next — Ch. 2, Generative AI at the Edge: building llama.cpp from source, running Qwen3.5 from the CLI and a local llama-server, and calling it from Python.

Questions?

Prof. Marcelo J. Rovai

rovai@unifei.edu.br

UNIFEI — Federal University of Itajubá, Brazil

AiEng4D — Academic Network Co-Chair

EdgeAIP — Academia-Industry Partnership Co-Chair

